There are many variants of this two-sum problem. Instead of asking for the sum to be equal to K, how will you handle the case if we want the maximum sum that is less than K? Basically, find two elements A[i] and A[j] such that their sum S is the maximum among all sums less than K. Note that we can't use the hash map in this case since we are NOT checking for the exact value. However, we can modify the above approach of sorting the array first and using the two pointers. I will leave it to our readers as an exercise to solve this problem.

Let us now consider the three-sum problem. Given an array of N integers, and a value K, are there triples (a,b,c) in the array such that a + b + c = K. If so, return all such triples; else, return an empty list. As usual, let us start with the brute force approach. We can consider all possible triples of the index values (i, j,k) and check whether the condition is satisfied for each of these combinations. What would be the time complexity of this algorithm? Since each index can run from 0 to N-1, we end up with O(N^3) complexity. Given that two-sum problems can be solved in O(N), we expect to be able to solve the three-sum problem in O(N^2)? Can we use the two-sum solution for solving the three-sum problem?

We can now reframe the three-sum problem in the following way. For each index value 'i', let 'c' = A[i]. Now, given 'c' and K, we need to find two elements in A such that their sum is equal to K-c. This is nothing but the two-sum problem that we know how to solve in O(N). Therefore, we can solve the three-sum problem in O(N^2). I will leave it to our readers to code up this solution. Another variant is to consider a three-sum problem, for which we want to find the triplet that has the maximum sum that is less than K.

The key point to realise based on our discussion is the following: for each new problem, start with a brute force solution. See where the time is getting spent. Look for a way to optimise the time. In many cases, it will be a trade-off, where we have to use up extra storage space to reduce the time complexity. Alternatively, you can think about solving the problem for an easier variant. For instance, instead of an unsorted array of integers, if the input array is sorted, how would you solve the two-sum problem? This immediately leads us to the two-pointer approach, and the O(NlogN) solution to solve the two-sum problem on an unsorted array of integers.

In many of the algorithmic problems, the key is to understand which part of the brute force solution can be modified to find an efficient solution. This mainly comes with practice.

A good resource for solving programming problems is the book 'Programming Interviews Exposed'. This book discusses the different topics that are typically covered during coding interviews, starting off from basic data structures such as arrays, linked lists, queues, stacks and binary search trees to recursion, back tracking, divide and conquer, dynamic programming, etc. The book provides a set of practice problems and discusses how many of these can be solved, step by step. This is a pretty useful book in addition to another title, 'Cracking the coding interview'.

Here is a simple problem for our readers to try their hands on. Most of you would be familiar with 'merge sort'. This is a 'divide and conquer' approach, whereby we split the input sequence into two equal parts, sort each of the individual sub-sequences, and then combine the two sorted sub-sequences. The merge part of the solution requires you to merge two sorted arrays into a single array. Merge sort typically uses additional O(N) extra storage to perform the merge. Here is the takeaway problem for this month.

1. Can you modify the merge sort so that you perform the sort in place with only constant additional storage?
2. Here is an extension of the original merge problem. Instead of two sorted arrays, you are given K sorted arrays. You need to create a single sorted array that is the combination of all sorted arrays.
3. This is a variation of Problem (2) above. Instead of creating a combined sorted array, you need to find the smallest ten elements of all the sorted arrays. Essentially, you need to return the first ten elements of the combined sorted array. But can you do it more efficiently than the solution for (2) above?
4. If you are asked to find both the smallest and the largest ten elements of all the sorted arrays, how would your solution change? What is the time complexity and space complexity of your approach?

Note that for all the above questions, each of the K sorted arrays is of different lengths. There can be duplicate elements across the arrays as well as in individual arrays.

Feel free to reach out to me over LinkedIn/email if you need any help in your coding interview preparations. If you have any favourite programming questions/software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at *sandyasm_AT_yahoo_DOT_com*.

I request our readers to follow the public health guidelines and stay safe. Please don't worry if you are not able to focus fully on learning new technologies, programming, etc. These are difficult times. So first take care of your mental and physical health, and help others wherever you can. Wishing all our readers happy coding until next month! Stay healthy and stay safe. END

By: Sandya Mannarswamy

The author is an expert in natural language processing and is currently working as an independent researcher. Her interests include natural language processing, machine learning and AI.